

Coverity 2026.6 Red Hat OpenShift deployment

Architecture, installation, migration, verification, and operational procedures for Coverity Connect.

Product release: 2026.6.0 · Document revision: 23 September 2026

Contents

- | | |
|-------------------------------|------------------------------------|
| 1. Deployment architecture | 7. Configure OpenShift routing |
| 2. Prerequisites and planning | 8. Verify the deployment |
| 3. Download the Helm chart | 9. Migration and upgrades |
| 4. Prepare secrets and trust | 10. Optional component planning |
| 5. Configure Helm values | 11. Operations and troubleshooting |
| 6. Validate and install | 12. References |

1. Deployment architecture

1.1 Full deployment reference architecture

Coverity can combine centralized results management, in-cluster analysis, analysis caching, and optional AI-Assisted Triage. The following logical architecture shows the wider platform and its supporting services. Components are enabled according to the deployment requirements.

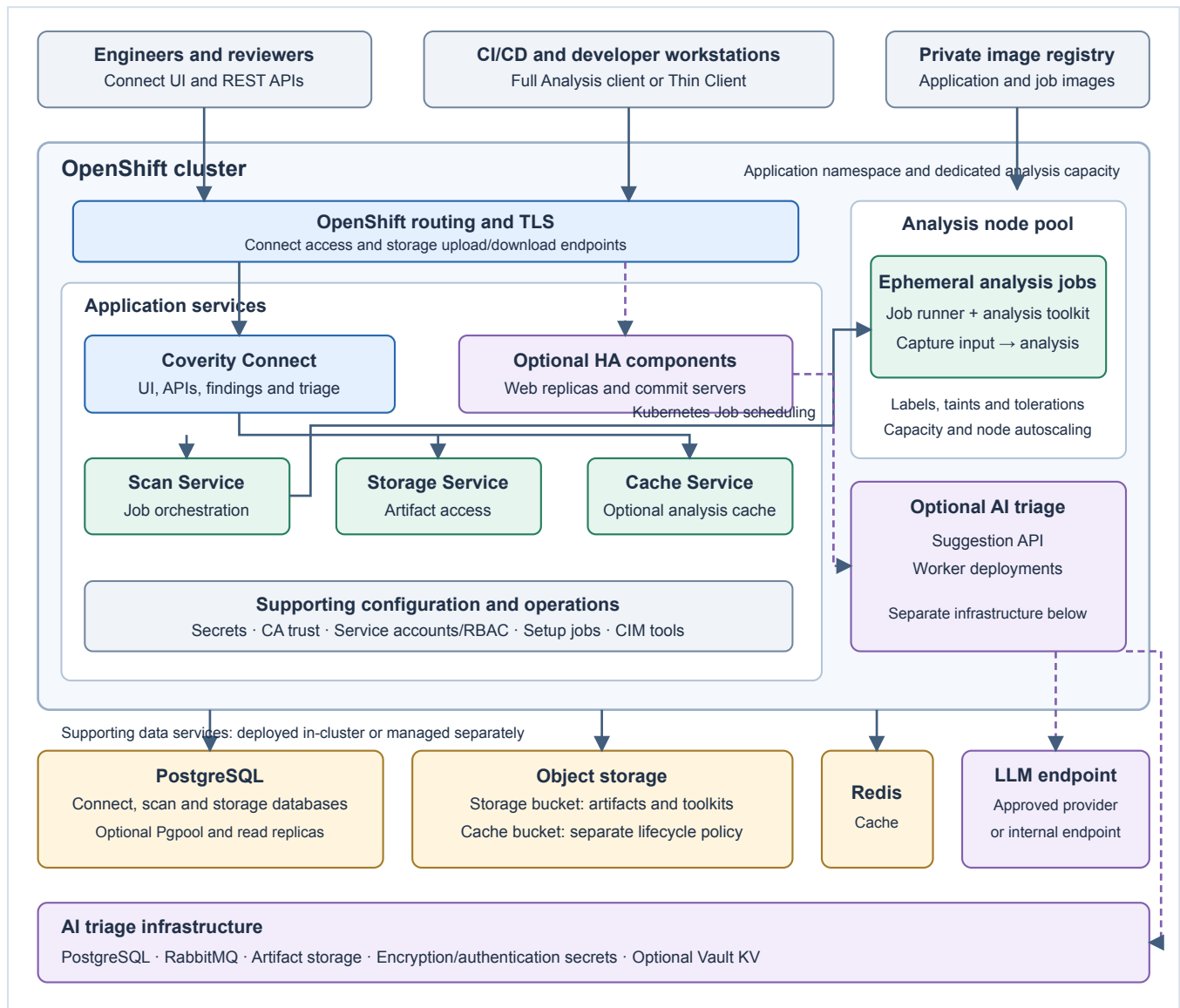


Figure 1. Full deployment reference architecture. Connections show principal service relationships, not an exhaustive network-policy matrix. Analysis jobs access artifacts and commit results to Connect. Purple components are optional deployment branches.

Deployment combinations: This diagram is a component reference, not a requirement to enable every feature together. The 2026.6 release notes identify an AI-Assisted Triage limitation with multiple Connect replicas. Review that limitation before combining AI triage and Connect HA.

1.2 Connect-only architecture used by this runbook

The procedures in sections 2 through 8 implement only the configuration below. Scan Services, Redis, object storage, RabbitMQ, Vault, AI triage, Pgpool, and additional Connect replicas are not deployed.

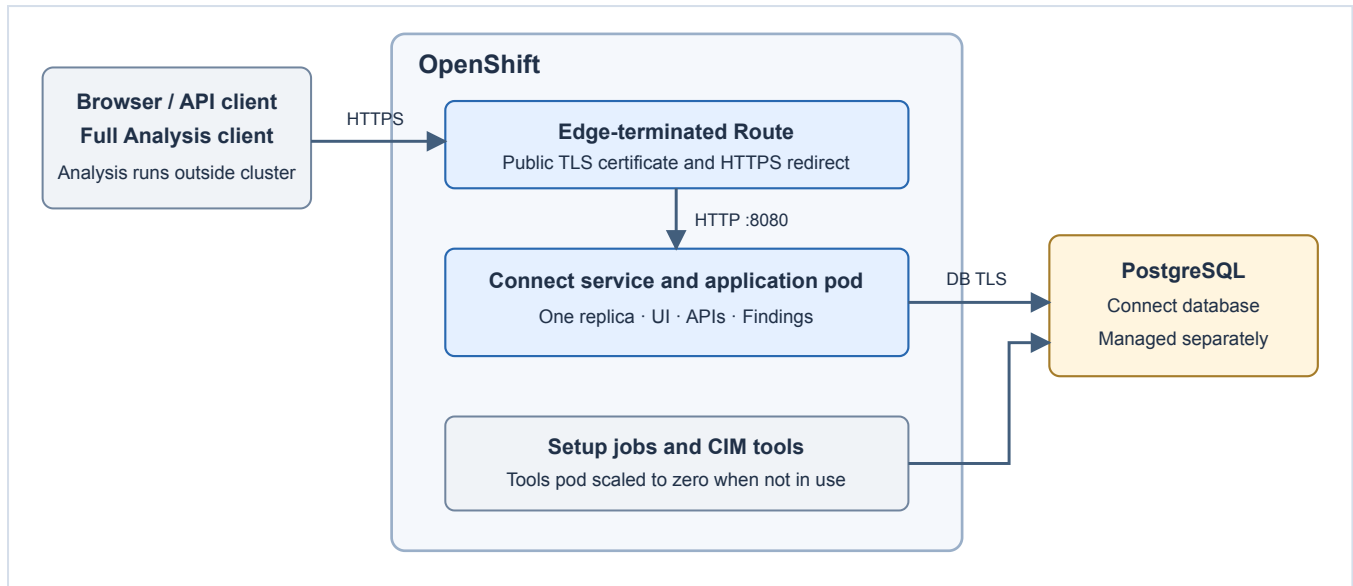


Figure 2. Connect-only deployment. The PostgreSQL database is external to the CNC release and may be hosted inside or outside OpenShift. The public Route is managed separately from Helm.

Edge termination encrypts client-to-router traffic. Router-to-Connect traffic uses HTTP. Environments requiring encryption on both connections need a re-encrypt Route, a working backend TLS listener, and a destination CA.

This single-replica configuration requires a maintenance window for upgrades and restarts. The Connect deployment uses the chart's Recreate strategy.

2. Prerequisites and planning

- A supported OpenShift cluster with the `route.openshift.io/v1` API available.
- Compatible `oc` and `Helm` clients. The CNC chart declares Kubernetes 1.31 or later as a chart minimum. Select the actual platform combination using the Coverity support matrix.
- Namespace permissions for deployments, StatefulSets, jobs, services, Secrets, ConfigMaps, and Routes.
- An active node-unlocked Connect license and application-registry credentials.
- A precreated PostgreSQL database and role with the required ownership, extensions, encoding, collation, and schema-upgrade permissions.
- A Connect DNS hostname of at most 46 characters and a matching TLS certificate.
- Network access to PostgreSQL, DNS, the image registry, and approved integrations.

Capacity

Set Connect resources from database size, user concurrency, commit activity, and the Connect sizing guidance. Include capacity for setup jobs, init containers, and administrative tools.

The PostgreSQL guidance specifies at least 8 CPUs and 32 GB RAM, with approximately 15% of database size as a RAM-planning guideline. Account for storage latency, IOPS, WAL, backup space, and growth. Confirm the final resource allocation through representative workload testing.

Working directory and variables

Use Bash in a dedicated deployment working directory. Replace example values before running commands and retain the same variables throughout the procedure.

Bash

```
export NS="coverity"
export RELEASE="coverity"
export VERSION="2026.6.0"
export CONNECT_HOST="coverity.example.com"
export DB_HOST="postgres.example.internal"
export DB_PORT="5432"
export DB_USER="coverity"

oc config get-contexts
oc config use-context "<target-context>"
oc whoami
oc version
helm version
oc api-versions | grep route.openshift.io

umask 077
mkdir -p charts secrets review
chmod 700 secrets review
```

If the project does not already exist, create it. Otherwise, use the existing approved project.

Bash

```
oc new-project "$NS"
oc get resourcequota,limitrange -n "$NS"
oc auth can-i --list -n "$NS"
```

3. Download the Helm chart

CNC chart: `oci://repo.blackduck.com/cloudnative/cnc`

Application images: `repo.blackduck.com/containers`

Bash

```
helm pull oci://repo.blackduck.com/cloudnative/cnc \
  --version "$VERSION" \
  --destination ./charts

helm show chart "./charts/cnc-$VERSION.tgz"
tar -xzf "./charts/cnc-$VERSION.tgz" -C ./charts

export CHART="./charts/cnc"

helm dependency list "$CHART"
helm show values "$CHART" > review/chart-defaults.yaml
```

Record the archive checksum. On Linux:

Bash

```
sha256sum "./charts/cnc-$VERSION.tgz" > review/chart-sha256.txt
```

On macOS, use `shasum -a 256` instead.

Release package: Preserve the dependencies included in the package. Resolve missing dependencies or incompatible metadata before installation. Do not refresh dependencies automatically with `helm dependency update` or `helm dependency build`.

Application images

Verify access to the following OpenShift images, or their approved mirrors:

- `repo.blackduck.com/containers/cim-web:2026.6.0-ubi`
- `repo.blackduck.com/containers/cim-tools:2026.6.0-ubi`
- `repo.blackduck.com/containers/cim-downloads:2026.6.0-ubi`

For a private mirror, preserve image names and tags and update `global.imageRegistry`. Configure registry CA trust on the cluster nodes separately from image-pull credentials.

4. Prepare secrets and trust

Place the following files in `secrets/`. Exclude this directory and rendered secret manifests from source control.

File	Purpose
registry-config.json	Docker authentication JSON with usable credentials, not just a workstation credential-helper reference.
db-password	Exact database password without an unintended trailing newline.
license.dat	Active Connect license.
admin-password	Fresh installation administrator password, without an unintended trailing newline.
postgres-root.pem	Trusted PostgreSQL CA bundle.
route.crt, route.key	Connect serving certificate and matching PEM private key.
ldap-key	Base64-encoded encryption key. For migration, the original <code>cim.ldap.key</code> value.

Create namespace-local resources:

Bash

```
oc create secret generic cnc-registry \
  --type=kubernetes.io/dockerconfigjson \
  --from-file=.dockerconfigjson=secrets/registry-config.json \
  -n "$NS"

oc create secret generic cnc-postgres \
  --from-literal=host="$DB_HOST" \
  --from-literal=port="$DB_PORT" \
  --from-literal=username="$DB_USER" \
  --from-file=password=secrets/db-password \
  -n "$NS"

oc create secret generic cnc-license \
  --from-file=license.dat=secrets/license.dat \
  -n "$NS"

oc create configmap cnc-trust \
  --from-file=postgres-root.pem=secrets/postgres-root.pem \
  -n "$NS"
```

Fresh installation only

Create the administrator Secret. Generate a unique LDAP encryption key only if this is a new installation and no key has already been prepared.

Bash

```
oc create secret generic cnc-admin \
  --from-file=password=secrets/admin-password \
  -n "$NS"

# Refuse to overwrite an existing encryption key.
(
  set -o noclobber
  openssl rand -base64 32 > secrets/ldap-key
)
chmod 600 secrets/ldap-key
```

Migration: Preserve the original `cim.ldap.key`. Do not generate a replacement. Leave `cim.cimweb.adminPasswordSecret` empty unless an administrator password reset is part of the approved migration procedure.

5. Configure Helm values

Create `site-values.yaml` from the example below. Replace every `REPLACE_...` entry. YAML values do not expand shell variables; enter the actual hostname and resource quantities.

Pass this file directly to the CNC chart. Do not add a `cnc:` wrapper around its contents.

YAML: `site-values.yaml`

```
global:
  imageRegistry: "repo.blackduck.com/containers"
  imageVersion: "2026.6.0"
  imageTagSuffix: "-ubi"
  imagePullSecret: "cnc-registry"
  licenseSecretName: "cnc-license"

  keygen:
    enabled: false

  postgres:
    database: "postgres"
    existingSecret: "cnc-postgres"
    sslmode: "verify-ca"

  trust-stores:
    enabled: true
    configmapName: "cnc-trust"

  ingress:
    enabled: false
    hosts:
      - "REPLACE_CONNECT_HOST"

  proxy:
    enabled: false

onPrem:
  postgres: false
  redis: false
  minio: false

scan-services:
  enabled: false

triage-suggestion-service:
  enabled: false

cnc-db-admin:
  enabled: false

cim:
  automountServiceAccountToken: false

  podSecurityContext:
    runAsNonRoot: true
    seccompProfile:
      type: RuntimeDefault

  postgres:
    database: "cim"
    existingSecret: "cnc-postgres"
```

```
    sslmode: "verify-ca"

cimweb:
  enabled: true
  replicas: 1
  webUrl: "https://REPLACE_CONNECT_HOST"
  contextPath: ""
  exposeCommitPort: false
  exposeMetrics: false

  adminPasswordSecret: "cnc-admin"

  updateLicense:
    enabled: true
    force: false

  loadBalancer:
    trustedRegex: 'REPLACE_TRUSTED_PROXY_REGEX'

  tlsSidecar:
    enabled: false

  resources:
    requests:
      cpu: "REPLACE_CPU_REQUEST"
      memory: "REPLACE_MEMORY_REQUEST"
    limits:
      cpu: "REPLACE_CPU_LIMIT"
      memory: "REPLACE_MEMORY_LIMIT"

cimtools:
  enabled: true
  automountServiceAccountToken: false

setupJob:
  enabled: true
  createDatabases: false
  activeDeadlineSeconds: 3600
  resources:
    requests:
      cpu: "REPLACE_SETUP_CPU_REQUEST"
      memory: "REPLACE_SETUP_MEMORY_REQUEST"
    limits:
      cpu: "REPLACE_SETUP_CPU_LIMIT"
      memory: "REPLACE_SETUP_MEMORY_LIMIT"

commit-server:
  replicas: 0

pgpool:
  enabled: false

ingress:
  enabled: false

route:
  enabled: false

gateway:
  enabled: false
  create: false

minioGateway:
```

```
gateway:  
  enabled: false
```

Configuration notes

- Use Kubernetes resource quantities, such as CPU "2" or "500m" and memory "8Gi". Select values for the workload rather than copying example quantities as capacity recommendations.
- Set the Connect database name explicitly with ordinary quotation marks.
- `createDatabases: false` requires a precreated database. It does not prevent schema initialization or upgrade.
- Keep `global.ingress.hosts` populated. Connect uses it to build its commit URL even when Ingress creation is disabled.
- Restrict the trusted proxy regex to source addresses observed by Connect. For example, `^10[.]20[.]1[.](10|11)$` matches two specific addresses without backslash escaping. Do not use an unrestricted `.*`.
- Let OpenShift SCC admission assign permitted user IDs. Do not routinely grant `anyuid` or privileged access.
- The Route is created separately after Connect starts successfully.

6. Validate and install

6.1 Render and review

Set KUBE_VERSION to the server version reported by `oc version`. The placeholder search should return no matches before continuing.

Bash

```
export KUBE_VERSION="REPLACE_SERVER_VERSION"

grep -n 'REPLACE_' site-values.yaml

helm lint "$CHART" \
  -f site-values.yaml \
  --set-file cim ldap.cimKey.value=secrets/ldap-key

helm template "$RELEASE" "$CHART" \
  --namespace "$NS" \
  --kube-version "$KUBE_VERSION" \
  --api-versions security.openshift.io/v1 \
  --api-versions route.openshift.io/v1 \
  -f site-values.yaml \
  --set-file cim ldap.cimKey.value=secrets/ldap-key \
  > review/rendered.yaml

chmod 600 review/rendered.yaml

oc apply --dry-run=server \
  -n "$NS" \
  -f review/rendered.yaml
```

Review images, resource allocations, database Secret references, security contexts, and pre-install jobs. The base configuration must not create Scan Services, bundled databases, Ingress, or Gateway resources.

Rendered manifests include sensitive configuration. Keep them protected. Install with Helm, not by applying the rendered file directly. Helm manages hook ordering. Server dry-run does not execute setup jobs or verify database connectivity and application behavior.

6.2 Install a new release

For a restored database, complete the migration preparation in section 9 first. The setup hook can modify the database schema.

Bash

```
helm install "$RELEASE" "$CHART" \
  --namespace "$NS" \
  -f site-values.yaml \
  --set-file cim ldap.cimKey.value=secrets/ldap-key \
  --wait \
  --timeout 60m
```

In a second terminal with the same variables, watch the workloads. Press Ctrl+C to stop watching before running further commands.

Bash

```
oc get pods,jobs -n "$NS" -w
```

Inspect setup results and application readiness:

Bash

```
oc logs -n "$NS" "job/$RELEASE-cim-setup"

oc rollout status "deployment/$RELEASE-cim" \
  -n "$NS" --timeout=15m

helm status "$RELEASE" -n "$NS"
oc get jobs -n "$NS"

# Fresh installation with the administrator password hook enabled:
oc logs -n "$NS" "job/$RELEASE-cim-password-setup"
```

7. Configure OpenShift routing

Create the public Route after Connect is ready. Point `CONNECT_HOST` to the intended router endpoint through DNS.

Bash

```
oc create route edge "$RELEASE-public" \
  --namespace "$NS" \
  --service="$RELEASE-cim" \
  --port=p8080 \
  --hostname="$CONNECT_HOST" \
  --cert=secrets/route.crt \
  --key=secrets/route.key \
  --insecure-policy=Redirect

oc annotate route "$RELEASE-public" -n "$NS" \
  haproxy.router.openshift.io/timeout=600s \
  --overwrite

oc get route "$RELEASE-public" -n "$NS"

oc get route "$RELEASE-public" -n "$NS" \
  -o 'jsonpath={.status.ingress[*].conditions}{"\n"}'
```

Add `--ca-cert=<chain-file>` when required by the PKI configuration. Adjust the timeout for representative uploads and commits.

Certificate renewal: This separately managed Route embeds certificate material. Renew it through the approved certificate-management process. Updating a separate Kubernetes TLS Secret does not renew this Route.

Connection	Required access
Clients to router	TCP 443; TCP 80 for HTTPS redirect.
Router to Connect	TCP 8080 for this edge-terminated configuration.

Connection	Required access
Connect, setup jobs and tools to PostgreSQL	Configured database TCP port with TLS.
Workloads to DNS	UDP and TCP 53 to cluster DNS.
Cluster nodes to registry	Registry HTTPS and trusted registry CA.
Connect to integrations	Only approved LDAP, SMTP, Jira and other integration destinations.

Define NetworkPolicy selectors for the actual router and DNS topology. Image pulls originate from nodes and are not controlled by ordinary pod NetworkPolicy.

8. Verify the deployment

Workloads and endpoints

Bash

```
oc get deployments,statefulsets,pods,jobs,services -n "$NS"

oc get endpointslices.discovery.k8s.io -n "$NS" \
  -l "kubernetes.io/service-name=$RELEASE-cim"

oc logs "deployment/$RELEASE-cim" -n "$NS" \
  -c cim-webapp --tail=200

oc get pods -n "$NS" \
  -o 'custom-columns=NAME:.metadata.name,SCC:.metadata.annotations.openshift\.io/scc'
```

The Connect deployment is <release>-cim and its application container is cim-webapp. The <release>-cim-tools StatefulSet starts at zero replicas.

HTTPS and redirect behavior

Bash

```
curl --fail --silent --show-error \
  --output /dev/null \
  --write-out 'HTTPS status: %{http_code}\n' \
  "https://$CONNECT_HOST/login/login.htm"

curl --silent --show-error \
  --output /dev/null --dump-header - \
  "http://$CONNECT_HOST/login/login.htm"
```

Verify hostname, certificate chain, and HTTP-to-HTTPS redirect. Use `--cacert` for an approved private CA rather than disabling certificate verification.

Application acceptance

1. Log in as administrator and verify version and license.
2. Verify ordinary-user access and project permissions.
3. Test LDAP or SAML, including canonical URL and redirect behavior.
4. Commit representative results from a full Coverity Analysis client.
5. Verify the resulting snapshot, findings, source browsing and cross-references.
6. Test configured email, Jira and other integrations.
7. Review pod restarts, OOM events, database errors and commit duration.

Retain the results with the deployment change record.

9. Migration and upgrades

Migration preparation

1. Record source Coverity and PostgreSQL versions, database size, authentication, integrations and custom properties.
2. Confirm the supported path to 2026.6.0. PostgreSQL major-version migration and Coverity schema upgrade are separate operations.
3. Back up the database and preserve the original `cim.ldap.key`, certificates and non-database configuration.
4. Restore a backup into an isolated target using the DBA-approved process. Restrict outbound integrations and production user access during rehearsal.
5. Set `cim.cimweb.adminPasswordSecret: ""` unless an administrator reset is intended. Review password requirements for upgrades from older releases.
6. Install against the restored database and verify schema upgrade, historical data and application behavior.
7. Measure restore duration, upgrade duration and peak resource usage.

Production cutover

1. Pause pipelines, commits, API writers and scheduled integrations.
2. Stop or quiesce the source application and verify that writers have stopped.
3. Take the final backup, restore the target and run the rehearsed upgrade and startup sequence.
4. Complete verification before switching Route or DNS traffic.
5. Resume selected pipelines, verify commits and then resume normal operation.

Recovery: Helm rollback does not reverse schema changes. Do not point older binaries at an upgraded database. Recover using a compatible backup and application version. Once the target accepts new writes, switching back requires an explicit decision about those writes and the recovery point.

Upgrading an existing Helm release

Export configuration to protected storage and prepare complete reviewed values for the new chart. Back up the database and stop all writers before the upgrade. The following commands apply to the Connect-only topology.

Bash

```
helm get values "$RELEASE" -n "$NS" \
  > review/current-values.yaml

helm get manifest "$RELEASE" -n "$NS" \
  > review/current-manifest.yaml

oc scale "deployment/$RELEASE-cim" \
  -n "$NS" --replicas=0

# Confirm application pods have stopped before proceeding.
oc get pods -n "$NS"

helm upgrade "$RELEASE" "$CHART" \
  --namespace "$NS" \
  --reset-values \
  -f site-values.yaml \
  --set-file cim.ldap.cimKey.value=secrets/ldap-key \
  --wait \
  --timeout 60m
```

Include every required site setting in the reviewed values file. Keep optional components explicitly disabled unless approved. A nonempty `adminPasswordSecret` runs the password setup hook again on upgrade; remove the reference after initial setup unless a reset is intended.

For HA and Scan Services, use the component-specific shutdown sequence. Scaling only the base Connect deployment does not stop every writer.

10. Optional component planning

Figure 1 shows the full platform. The following requirements support a later expansion beyond the Connect-only installation.

Scan Services and analysis nodes

- Configure Scan Service and Storage Service databases with explicit names.
- Provide a separate Analysis license when the Connect license does not also license analysis execution.
- Provision object storage and configure its adapter, bucket, credentials, internal and external endpoints, and CA trust.
- Do not configure provider expiry rules on the Storage Service bucket. Use application retention for scan data.
- Define analysis-job node labels, taints, tolerations, resource sizes and autoscaling. Verify placement on an actual analysis Job.
- Configure Kubernetes service accounts and RBAC for job orchestration.
- Enabling Scan Services also enables the Connect TLS sidecar through chart logic. Verify its image, certificates, internal TLS and upload limits.
- Upload required toolkits and verify capture upload, analysis, result commit and artifact download before production use.

Analysis caching

- Provide Redis and a cache bucket separate from the Storage Service bucket.
- Create the bucket before installation. Configure cache expiry of at least 8 days; the documented recommendation is 30 days.
- Configure Redis authentication, TLS and hostname verification.
- Cache Service expects a password Secret key named password. The bundled Redis chart normally uses redis-password. Align these credential contracts explicitly.
- Select maintained images compatible with the chart and required modules.

Connect HA and Pgpool

- Plan application and commit-server replicas, capacity, placement, disruption handling and database connections.
- Account for the commit-server template's Ingress creation, which is independent of the ordinary Ingress enable flags.
- Route /ccd to the correct commit backend. Pgpool deployments require additional routing for relevant write operations.
- Review Connect-to-Pgpool traffic separately: the chart helper disables PostgreSQL TLS on that connection.
- Provide explicit primary and replica addresses for Pgpool configuration.
- Keep separate Connect releases in separate namespaces.

AI-Assisted Triage

- Approve transmission of source context and findings to the selected LLM.
- Configure the triage database, RabbitMQ, artifact storage, encryption keys and authentication secrets.

- Verify authentication file mounts in both API and worker pods.
- Verify actual service names, upload URLs, credentials, TLS and network access.
- Do not deploy development-mode Vault or fixed development root tokens in production. Use an approved secrets-management design if Vault is needed.
- Review effective retention and telemetry settings.
- Resolve the release limitation involving multiple Connect replicas before combining AI triage with HA.

11. Operations and troubleshooting

Symptom	Investigation
Setup failure or timeout	Inspect job logs and pod events. Check database connectivity, permissions, Secret keys, CA trust, schema version and resources.
ImagePullBackOff	Check image path, UBI tag, architecture, registry credentials, connectivity and registry CA trust.
SCC admission denied	Inspect ReplicaSet or Job events and pod security contexts. Correct the specific issue before considering a narrowly scoped exception.
Read-only filesystem error	Identify the write path and required writable mount. Use <code>/workdir</code> for tools output.
Route 503	Check admission, pod readiness, service endpoints, target port and NetworkPolicy.
SAML redirect errors	Check canonical web URL, DNS, forwarded headers and trusted proxy configuration.
Commit/upload failure	Check router timeouts, upstream limits, application logs and database performance. Include sidecar and object storage paths when Scan Services is enabled.

Bash: diagnostics

```
oc get events -n "$NS" --sort-by=.metadata.creationTimestamp
oc describe deployment "$RELEASE-cim" -n "$NS"
oc describe job "$RELEASE-cim-setup" -n "$NS"

oc logs "deployment/$RELEASE-cim" \
  -n "$NS" -c cim-webapp --tail=200
```

Database integrity check

Scale up the tools pod and open its administration shell:

Bash

```
oc scale "statefulset/$RELEASE-cim-tools" \
  -n "$NS" --replicas=1

oc rollout status "statefulset/$RELEASE-cim-tools" \
  -n "$NS" --timeout=5m

oc exec -it -n "$NS" \
  "statefulset/$RELEASE-cim-tools" \
  -- /coverity/shell-entrpoint.sh
```

Inside the tools shell:

Bash

```
check-integrity.sh
exit
```

Save required logs and reports before scaling down:

Bash

```
oc scale "statefulset/$RELEASE-cim-tools" \
  -n "$NS" --replicas=0
```

Temporary files: /workdir is an emptyDir volume and is lost when the pod is deleted. Copy required files to protected external storage first. Configure the optional /data PVC for persistent tools storage.

Ongoing responsibilities

- Schedule database backups and periodic restoration tests.
- Preserve the LDAP encryption key and non-database configuration.
- Assign owners for certificate and license renewal, images and capacity.
- Collect logs outside pods and monitor restarts, OOMs, throttling, database latency, storage growth and commit duration.
- Review static PostgreSQL tuning after resource changes. Start with tuning suggestions, obtain DBA approval, and schedule any required restart.

12. References

- CNC Helm chart: [oci://repo.blackduck.com/cloudnative/cnc](https://repo.blackduck.com/cloudnative/cnc)
- Application registry: repo.blackduck.com/containers
- [Black Duck Community](https://community.blackduck.com/) (<https://community.blackduck.com/>)
- [Helm installation](https://helm.sh/docs/intro/install/) (<https://helm.sh/docs/intro/install/>)
- [OpenShift documentation](https://docs.redhat.com/en/documentation/openshift_container_platform) (https://docs.redhat.com/en/documentation/openshift_container_platform)

Relevant sections of the **Coverity 2026.6 Cloud Deployment Administrator and User Guide**:

- About Coverity cloud deployment

- Third-party software and platform support matrix
- Downloading the Helm chart from the Black Duck public Docker registry
- Red Hat OpenShift infrastructure and configuration
- PostgreSQL databases and PostgreSQL pod
- Sizing a Coverity Connect (cim) pod for optimum performance
- Configuring pod and container security
- Creating scan job node pools and scheduling scan jobs
- Upgrading a Coverity cloud deployment
- Coverity tools in a Coverity cloud deployment

Coverity 2026.6 OpenShift Engineering Runbook.

Retain the release package, reviewed configuration, backup references and verification results with the deployment change record.